Guidelines for Implementation
SPINE-IoT
Use Case "Flexible Start for White Goods"

Summary

This document aims to support Energy Management System providers and Smart Device manufacturers in their implementation of the EN50631 Standard - SPINE-IoT Connectivity Solution.

It describes the use of interface between an Energy Manager API and a device API from the Energy manager API point of view.

It includes detailed procedures for each phase, error handling, minimum requirements for a basic implementation, and also a section of Frequently Asked Questions.

For the use of interface, it outlines a three-phase process:

1. Get Interface Access: Describes obtaining the necessary keys to access devices via the API (cloud representations).
2. Get Overview: Details how to retrieve information about devices and their capabilities (complete list of all or individual devices and list of use cases for all or individual devices)
3. Setup and Maintain Subscription and Binding: Explains how to subscribe to devices and entities to receive updates, and handle subscriptions and bindings.

Glossary of Terms

CCM – Customer Connectivity Manager / EMS - Energy Management System

ESA – Energy Smart Appliance

CCM / EMS Cloud – Customer Connectivity Manager / Energy Management System Cloud

ESA Cloud – Energy Smart Appliance Cloud / Appliance Manufacturer Cloud

FSWG – Flexible Start for White Goods

JKW – JSON Key Website

JKWS – JSON Key Website Sets

JWT – JSON Web Token

Contents

1 Get Access to the Interface 3
2 Get Overview and FSWG Scenarios 1 and 2 3
2.1 GET /devices 4
2.1.1 All devices 4
2.1.2 A specific Device 5
2.2 GET /entities 5
2.2.1 All Devices and Entities 6
2.2.2 A specific Device and/or a specific Entity 6
2.3 FSWG Scenario 1: Announcement of a Plan 8
2.4 GET /features 8
2.4.1 GET /features/powerSequence 9
2.4.2 GET /features/powerTimeSlot 10
2.4.3 GET /features/powerSequenceNode 12
2.4.4 GET /features/powerSequenceAlternativesRelation 12
2.5 GET /usecases 13
2.6 GET /usecaseInterfaces/flexibleStartForWhiteGoods/v1 13
2.7 FSWG Scenario 2: Shift preferred power sequence 14
3 Setup and maintain Binding and Subscription 15
3.1 Phase 3a - Enable access by creating Binding 16
3.1.1 Create a Binding 16
3.1.2 Renewal of Binding 17
3.1.3 Delete Binding 17
3.1.4 Retrieve active Bindings 18
3.2 Phase 3b - Enable Callbacks by creating a Subscription 18
3.2.1 Create a Subscription 18
3.2.2 Receive Callback (CCM) 19
3.2.3 Validation of Callback (CCM) ..... **Fehler! Textmarke nicht definiert.**
3.2.4 Error handling for subscriptions and callbacks 21
4 Common workflow 22
Annex 1 – Mandatory Implementation Requirements 23
Annex 2 – Error types 27
Annex 3 – Frequently Asked Questions 29

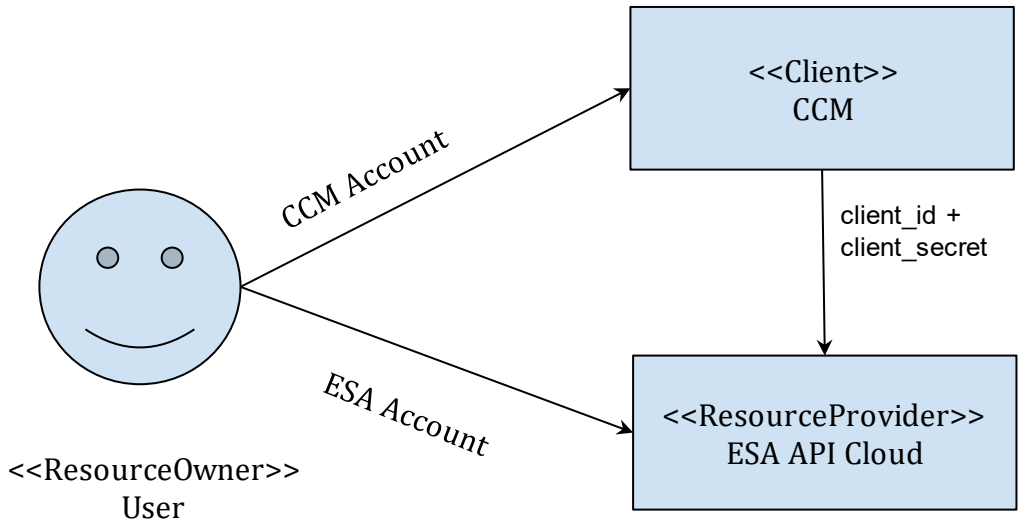
1 Get Access to the Interface

MANDATORY.
Details: Account linking is only possible with OAuth2 Authorization Code Grant Flow.

1. A CCM provider has to request a *client_id* and a *client_secret* (= client credentials) from the ESA provider. This is used to authenticate the client/application of the CCM in the following OAuth2 Flow.
2. A CCM provider can then trigger the OAuth2 Authorization Code Grant Flow to retrieve an *auth_code* and, eventually, an *access_token* and a *refresh_token* on behalf of a customer.

Note: You need to provide the access_token in the "Authentication"-header of every subsequent request to the EN50631 API. In our examples, this header is marked as "Authentication: provider-specific JWT as Bearer header".

More details can be found here:
https://www.rfc-editor.org/rfc/rfc6749
or here:
https://oauth.net/2/



2 Get Overview and FSWG Scenarios 1 and 2

The CCM wants to know what devices are available and what they can do.

- Full list of all or individual devices
- Use cases for all or individual devices
- If necessary, subscribe to devices and entities to receive changes to the device list (see chapter 3).

A Client must always specify a presumed major version, in the simplest case starting with "v1".

Example Request:

GET https://<api.esa.cloud>/v1/api
Authentication: provider-specific JWT as Bearer header

Example Response:
Start with Version 1.0.0


```
{
 "thisVersion": "1.0.0",
 "otherVersions": []
}
```


Example of what it could look like in the future:


```
{
 "thisVersion": "4.5.8",
 "otherVersions": [
  "2.3.1","3.2.5"
 ]
}
```


2.1 GET /devices

MANDATORY.

Details:

- Device Type und Entity Type according to EEBUS Spec (cf. EEBus SPINE TS - Resource Specification v1.3.0, pp. 660 and pp. 668 and pp. 72)

- A Device can have multiple Entities; Features are always bound on an Entity-level
  - Thus, the hierarchy can be described as follows: Device -> Entity -> Feature
  - See "EEBus UC Technical Specification Flexible Start for White Goods" section 3.1.5, Figure 6.
- deviceAddress is not needed because it is needed in SPINE and is only necessary if interaction with SPINE is required!
  - In SPINE-IoT, deviceID is used for this purpose.

2.1.1 All devices

MANDATORY.

Example Request:
GET https://<api.esa.cloud>/v1/devices
Authentication: provider-specific JWT as Bearer header

Example Response:

a. Example Dishwasher and Washer


```
[
 {
  "deviceId": "1234567890",
  "deviceType": "Washer ",
  "entitiesOverview": {
   "entityTypes": [
    "Washer"
```

```
      ]
    }
  },
  {
    "deviceId": "1234567891",
    "deviceType": "Dishwasher",
    "entitiesOverview": {
      "entityTypes": [
        "Dishwasher"
      ]
    }
  }
]
```


b. Example Dryer


```
[
  {
    "deviceId": "1234567893 ",
    "deviceType": "Dryer",
    "entitiesOverview": {
      "entityTypes": [
        "Dryer"
      ]
    }
  }
]
```


2.1.2 A specific Device

MANDATORY.

Example Request:
GET https://<api.esa.cloud>/v1/devices?deviceId=1234567890
Authentication: provider-specific JWT as Bearer header

Example Response:


```
[
 {
   "deviceId": "1234567890",
   "deviceType": "Washer ",
   "entitiesOverview": {
     "entityTypes": [
       "Washer"
     ]
   }
 }
]
```


2.2 GET /entities

MANDATORY.
Details:

- Device Type (only in the query) and Entity Type according to EEBUS Spec (see above)
- deviceAddress is not needed (see above)! See EN 50631-4-1:2023 (E) Table 2 on mandatory fields.
- Actors according to EEBUS UC TS:
  - SmartAppliance: "SmartAppliance" (cf. EEBus UC TS - Flexible Start for White Goods - 3.2.1.1, p. 31)

2.2.1 All Devices and Entities

MANDATORY.

Example Request:
GET https://<api.esa.cloud>/v1/entities
Authentication: provider-specific JWT as Bearer header

Example Response (Washer):


```
[
 {
  "deviceId": "1234567890",
  "entityId": 0,
  "entityType": "Washer",
  "usecasesOverview": [
   {
    "name": "flexibleStartForWhiteGoods",
    "usecaseMajorVersion": "v1",
    "actor": "SmartAppliance",
   }
  ],
  "featuresOverview": {
   "featureObjTypes": [
    "powerSequence",
    "powerTimeSlot",
    "powerSequenceNode",
    "powerSequenceAlternativesRelation"
   ],
   "featureTypes": [
    "PowerSequences"
   ]
  }
 }
]
```


2.2.2 A specific Device and/or a specific Entity

MANDATORY.

Example Request:
GET https://<api.esa.cloud>/v1/devices?deviceId=1234567890
Authentication: provider-specific JWT as Bearer header

OR

GET https://<api.esa.cloud>/v1/devices?deviceId=1234567890&entityId=0
Authentication: provider-specific JWT as Bearer header

OR

GET https://<api.esa.cloud>/v1/devices?entityType=Washer
Authentication: provider-specific JWT as Bearer header

Example Response:
Both examples from above lead to the same result (in this example of one washing machine within the user's ESA Cloud account). An entityId is only unique within a device.


```
[
 {
  "deviceId": "1234567890",
  "entityId": 0,
  "entityType": "Washer",
  "usecasesOverview": [
   {
    "name": "flexibleStartForWhiteGoods",
    "usecaseMajorVersion": "v1",
    "actor": "SmartAppliance",
   }
  ],
  "featuresOverview": {
   "featureObjTypes": [
    "powerSequence",
    "powerTimeSlot",
    "powerSequenceNode",
    "powerSequenceAlternativesRelation"
   ],
   "featureTypes": [
    "PowerSequences"
   ]
  }
 }
]
```

2.3 FSWG Scenario 1: Announcement of a Plan

MANDATORY.
Details: Scenario 1 and corresponding feature-objects must be available and supported. It might be unavailable via the respective /features-endpoint but available via /usecases and /useCaseInterfaces-endpoints.

The Use case FSWG uses several different Use Case Functions.

Mapping of User Stories, Use Cases and Use Cases Functions, see EN 50631-1:2023 (E):

Epic	Use Cases	Use Cases Functions
Appliance Energy Flexibility Epic 1,2	Flexible Start for White Goods	UCF_Plan_With_Power_Sequences UCF_Select_Power_Sequence UCF_Shift_Preffered_Power_Sequence UCF_Configure_Current_Power_Sequence

The Use case functions can be mapped to different Scenarios as follows, see EN 50631-1:2023 (E):

Scenario	Primary Use Case Function
Announcement of plan	UFC_Plan_With_Power_Sequence
Shift preferred power sequence	UCF_Shift_Preffered_Power_Sequence
Select alternative power sequence	UCF_Select_Power_Sequence
Configure current power sequence	UCF_Configure_Current_Power_Sequence

Of these, only Scenario 1 and one of Scenarios 2 or 3 are MUST requirements (cf. EN 50631-1:2023 (E) - Table 4). In most cases, the announcement of a plan (scenario 1) and shifting a power sequence (scenario 2) will be available.

How you can use the Use case functions via REST API:

- Announcement of a plan / UCF_Plan_with_Power_Sequences (cf. EN 50631-1:2023 (E) - Table 5), i.e. the Announcement of a plan, are represented in the OpenAPI through various /features/…-endpoints
- Shifting a power sequence / UCF_Shift_Preferred_Power_Sequence is represented by a POST on /usecaseInterfaces/flexibleStartForWhiteGoods/v1 for shifting a power sequence.

The endpoints for these mandatory scenarios 1 and 2 are explained in the following. For now, scenario 3 is omitted in this cookbook.

2.4 GET /features

OPTIONAL.

Note: Data structures explained in this section are also used on other interfaces. This holds esp. for data structures in responses or callbacks of use cases. Readers are encouraged to briefly read these sections (even if a "/features" path is not supported, see Annex 1 – Mandatory Implementation Requirements) as this will help understanding other interfaces.
A "GET /features" delivers data of requested features. The response contains a list of objects with these elements:

- deviceId
- entityId
- featureObjType
- featureType
- data

In case of FSWG, "featureType" is always set to "PowerSequences". "featureObjType" always denotes the content type of "data". For FSWG, the following featureObjType values are used: "powerSequenceNode", "powerTimeSlot", "powerSequenceNode", "powerSequenceAlternativesRelation".
Data of a particular featureObjType can either be requested by "GET /features" with proper query parameter or by specific paths (see following sub-sections).
Please note that accessing FSWG data of a device/entity is preferred via the respective usecaseInterfaces path as described in section 2.6. A usecaseInterfaces path provides a more simple way of getting and modifying data.

As you can see above, the feature "PowerSequences" must be supported with the respective ObjectTypes:

```
"featuresOverview": {
   "featureObjTypes": [
    "powerSequence",
    "powerTimeSlot",
    "powerSequenceNode",
    "powerSequenceAlternativesRelation"
   ],
   "featureTypes": [
    "PowerSequences"
   ]
  }
```

2.4.1 GET /features/powerSequence

OPTIONAL.
Details: Endpoint is optional, object is MANDATORY due to usecaseInterfaces-endpoint. Refer to 2.6.

This endpoint provides basic information about a given sequence on an entity. Most importantly:

- If state is set to "scheduled": indicates to the CCM that this is the preferred powerSequence in EEBus terms, i.e. an Announcement of a Plan is active (see above)
- If state is set to "inactive": The CCM must expect that most data of a currently inactive power sequence won't be valid. This is true for this and all corresponding endpoints.

- With this version of the cookbook for FSWG, the maximum number of allowed powerSequences per device is one (1), i.e. a device won't return more than one powerSequence, regardless of the individual powerSequence's state. Thus, a powerSequence reselection is not supported. This is also reflected in the information of the powerSequenceAlternativesRelation-endpoint as no alternative to the currently selected sequenceId is provided.
- valueSource: indicates the accuracy the CCM can expect from the slot power data
- An appliance MAY support or not support pausing/resuming or stopping a power sequence.

Example Response:


```
[
 {
  "deviceId": "1234567890",
  "entityId": 0,
  "featureObjType": "powerSequence",
  "featureType": "PowerSequences",
  "data": {
   "sequenceId": 0,
   "description": "Whites",
   "powerUnit": "W",
   "valueSource": "calculatedValue",
   "state": "scheduled",
   "activeSlotNumber": 0,
   "sequenceRemoteControllable": true,
   "startTime": "2024-02-15T15:21:08.028Z",
   "endTime": "2024-02-15T15:21:08.028Z",
   "earliestStartTime": "2024-02-15T15:21:08.028Z",
   "latestEndTime": "2024-02-15T15:21:08.028Z",
   "isPausable": false,
   "isStoppable": false
  }
 }
]
```


2.4.2 GET /features/powerTimeSlot

OPTIONAL.
Details: Endpoint is optional, object is MANDATORY due to usecaseInterfaces-endpoint. Refer to 2.6.

- This endpoint provides the expected power consumption of the different slots, i.e. program parts. Most importantly: all fields power, powerMin and powerMax are optional. Accordingly, it shouldn't be expected that they are all filled. But at least one of the power fields SHALL be set.
- The granularity of a power sequence (i.e. its dissection into slots) is up to the appliance manufacturer; it might only provide one slot per sequence, i.e. very basic power information, or several slots per sequence, i.e. more sophisticated power information.

Please note that also phases of no power consumption during a power sequence must be represented by a slot (a power sequence must not have a gap).

Example Response:


```
[
 {
  "deviceId": "0123456789",
  "entityId": 0,
  "featureObjType": "powerTimeSlot",
  "featureType": "PowerSequences",
  "data": {
   "sequenceId": 0,
   "slotNumber": 0,
   "defaultDuration": "PT15M",
   "description": "Rinse",
   "power": {
    "number": 3,
    "scale": 2
   },
   "powerMin": {
    "number": 2,
    "scale": 2
   },
   "powerMax": {
    "number": 5,
    "scale": 2
   }
  }
 },
 {
  "deviceId": "0123456789",
  "entityId": 0,
  "featureObjType": "powerTimeSlot",
  "featureType": "PowerSequences",
  "data": {
   "sequenceId": 0,
   "slotNumber": 1,
   "defaultDuration": "PT1H15M",
   "description": "",
   "power": {
    "number": 8,
    "scale": 2
   }
  }
 }
```

]

2.4.3 GET /features/powerSequenceNode

OPTIONAL.
Details: Endpoint is optional, object is MANDATORY due to usecaseInterfaces-endpoint. Refer to 2.6.

This endpoint retrieves the power sequence node information of a device, including the device's general remote controllability and reselection support status.

Most importantly:

- supportsReselection: If set to "true," the power sequences server allows unlimited re-selections within an "alternatives" group. If set to "false," only one selection is permitted.

Example Response:


```
[
 {
  "deviceId": "1234567890",
  "entityId": 0,
  "featureObjType": "powerSequenceNode",
  "featureType": "PowerSequences",
  "data": {
   "nodeRemoteControllable": true,
   "supportsReselection": false
  }
 }
]
```


2.4.4 GET /features/powerSequenceAlternativesRelation

OPTIONAL.
Details: Endpoint is optional, but object is still necessary for full power plan announcement, even if no alternative is defined by ESA. Also refer to 2.6.

This endpoint retrieves the relationship between power sequence alternatives and the currently selected sequence for a device.

Most importantly:

- As only one powerSequence per device is supported for FSWG, most vendors will not provide alternative sequences.
- More than one (alternative) power sequence MAY be provided for a given device/entity address.

Example Response:


```
[
```

```
 {
   "deviceId": "string",
   "entityId": 0,
   "featureObjType": "powerSequenceAlternativesRelation",
   "featureType": "PowerSequences",
   "data": {
    "alternativesId": 0,
    "sequenceId": [
      0
    ]
   }
 }
]
```


2.5 GET /usecases

MANDATORY.

This endpoint indicates to the CCM which Scenarios of the FWSG use case are supported.

Example Response:


```
[
  {
     "deviceId" : "1234567890",
     "entityId" : 0,
     "usecase" :
     {
        "name": "flexibleStartForWhiteGoods”,
        "usecaseVersion": "1.0.0",
        "usecaseMajorVersion": "v1",
        "actor": "SmartAppliance",
        "scenarioSupport": [
           1,
           2
        ],
        "isOnThisEntity": true,
        "childEntities": []
     }
  },
]
```


2.6 GET /usecaseInterfaces/flexibleStartForWhiteGoods/v1

MANDATORY.

This endpoint is basically a convenient wrapper around all necessary objects for this use case, in our case all features-objects (see 2.4).

Example Response:


```
[
  {
    "deviceId": "1234567890",
    "entityId": 0,
    "featureObjType": "powerSequence",
    "featureType": "PowerSequences",
    "data": {
      ...
    }
  },
  {
    "deviceId": "1234567890",
    "entityId": 0,
    "featureObjType": "powerTimeSlot",
    "featureType": "PowerSequences",
    "data": {
      ...
    }
  },
  {
    "deviceId": "1234567890",
    "entityId": 0,
    "featureObjType": "powerSequenceNode",
    "featureType": "PowerSequences",
    "data": {
      ...
    }
  },
  {
    "deviceId": "1234567890",
    "entityId": 0,
    "featureObjType": "powerSequenceAlternativesRelation",
    "featureType": "PowerSequences",
    "data": {
      ...
    }
  }
]
```


2.7 FSWG Scenario 2: Shift preferred power sequence

MANDATORY.

Refer to 2.3 regarding the content of the different FSWG Scenarios.

This endpoint allows you to shift the preferred power sequence of a device by specifying a new start time in UTC.

Most importantly:

- The timezone used for specifying a new startTime is the standard UTC timezone, as specified by EN50631.

Example Request:
POST https://<api.esa.cloud>/v1/usecaseInterfaces/flexibleStartForWhiteGoods/v1
Authentication: provider-specific JWT as Bearer header

```
{
 "deviceId": "0123456789",
 "entityId": 0,
 "data": {
   "shiftSelectSequence": {
    "sequenceId": 0,
    "startTime": "2024-02-23T11:45:11.756Z"
   }
 }
}
```

Example Responses:
Example (Success):
HTTP/1.1 204

Example (Failure: Provided startTime is out of range, e.g. new endTime (startTime + duration) is later than latestEndTime):
HTTP/1.1 400
Content-Type: application/problem+json
Content-Language: <<isoLocale>>

```
{
 "type": "tag:ems.miele.com,2024-12-17:/errors/value-out-of-range#correlationId",
 "title": " Value out of Range",
 "status": 400,
 "detail": " ToDo",
 "instance": "ToDo"
}
```

3 Setup and maintain Binding and Subscription

3.1 Phase 3a - Enable access by creating Binding

3.1.1 Create a Binding

MANDATORY.
Details:

- Binding for usecaseInterfaces is MANDATORY. Binding for features is optional.
  - A binding request SHOULD contain deviceId and entityId. A CCM SHALL expect that binding requests without valid deviceId and entityId might be rejected by the ESA cloud.
    Rationale: Many use cases (like FSWG) require exactly one binding. Binding requests without deviceId/entityId are unspecific and might lead to unintended blocking of a particular usecaseInterface. This means another legitimate client might be unable to acquire a binding permission because an unspecific binding request unintendedly already got the binding permission.
- For a given appliance supporting FSWG, at most one CCM can be given FSWG control.
  - Consequence: Binding permission to the respective Use Case instance can be given to at most one authorized account.
    - Exception (unlikely; just for the sake of completeness): For some reason a user has redundant CCM instances (with individual identities) and ensures that these CCM instances are synchronized/do not interfere with each other AND the cloud provider provides such configuration. This exception is NOT considered further.
  - Consequence: IF the Use Case interface to this appliance is available for different accounts, the ESA cloud must ensure that only one account can get binding permission.

CCM → ESA Cloud

Example Request (1), useCaseInterface-Binding

POST https://<api.esa.cloud>/v1/bindings
Authentication: provider-specific JWT as Bearer header

```
{
 "usecaseInterfaces": {
  “deviceId”: “1234567890”,
  “entityId”: 0,
  "usecaseName": "flexibleStartForWhiteGoods",
  "usecaseMajorVersion": "v1,
  "actor": "SmartAppliance",
  "requester": {
   "actor": "CEM",
   "usecaseVersion": "1.0.0",
  }
 }
}
```

Example Response

```
{
 "bindingId": "67890",
 "expires": "2024-06-29T16:30:00Z"
}
```

3.1.2 Renewal of Binding

MANDATORY.

The renewal of a binding can be used to extend the binding before it expires. If it has already expired, a new binding has to be created as described above.

CCM → ESA Cloud
PATCH https://<api.esa.cloud>/v1/bindings
Authentication: provider-specific JWT as Bearer header

Example Request

```
{
 "renewAll": true
}
```

Example Response

```
[{
 "bindingId": "67890",
 "expires": "2024-06-30T16:30:00Z"
},
{
 "bindingId": "67891",
 "expires": "2024-06-30T16:30:00Z"
}
]
```

3.1.3 Delete Binding

MANDATORY.

The deletion of a binding can be used to delete binding before it expires.
CCM → ESA Cloud

Example Request

DELETE https://<api.esa.cloud>/v1/bindings?bindingId=abcd-1234

Authentication: provider-specific JWT as Bearer header

3.1.4 Retrieve active Bindings

MANDATORY.

CCM → ESA Cloud
GET https://<api.esa.cloud>/v1/bindings
Authentication: provider-specific JWT as Bearer header

No request params required to get active bindings

Example Response


```
[{
 "bindingId": "67890",
 "expires": "2024-06-29T16:30:00Z" },
{
 "bindingId": "67891",
 "expires":  "2024-06-29T16:30:00Z" }
]
```


3.2 Phase 3b - Enable Callbacks by creating a Subscription

CCM → ESA Cloud
POST https://<api.esa.cloud>/v1/subscriptions
Authentication: provider-specific JWT as Bearer header

3.2.1 Create a Subscription

MANDATORY.
Details:

- Subscription on usecaseInterfaces is mandatory as Bindings on usecaseInterfaces is also mandatory for FSWG use case.
- Support of subscription on /devices and /entities is mandatory. Rationale: This might add necessary information to prioritize all Use Case instances according to the relevance of the devices/entities.
- Support of subscription on /usecases is mandatory if the ESA cloud provides Use Cases (esp. via /usecaseInterfaces). Rationale: A CCM needs to be provided with updates in case of dynamically available Use Cases (example: EV is connected/disconnected to a wallbox). Note: Subscription on /usecaseInterfaces might not be sufficient to discover potentially available Use Case instances ahead of time.
    i) Note: ESA cloud providers might never send an update for the usecases-object as scenario support is not dynamic in context of FSWG. Thus, the content of the usecases-object always remains the same and CEM cannot expect any data to be sent when subscribing to the usecases-object.

- Support of subscription on /features is optional.

Example Request

POST https://<api.esa.cloud>/v1/subscriptions
Authentication: provider-specific JWT as Bearer header


```
{
"callbackUrl": "https://subscribingserver.com/receivecallbacks/particularCallback",
"usecases": {
   "deviceId": "1234567890",
   "entityId": 0,
   "usecaseName": "flexibleStartForWhiteGoods",
   "usecaseMajorVersion": "v1"
 },
 "devices": {
   "deviceId": "1234567891",
 },
 "entities": {
   "deviceId": "1234567892",
   "entityId": 0,
 },
 "usecaseInterfaces": {
   "deviceId": "12345678903",
   "entityId": 0,
   "usecaseName": "flexibleStartForWhiteGoods",
   "usecaseMajorVersion": "v1",
   "actor": "CEM"
 }
}
```


- Objects in the subscription request are independent from each other. In a subscription request you can have a "devices" object with deviceId "abc" and an "entities" object with deviceId "cde" and entityId 18. These objects are independent. They are just created with the same subscription request.

Example Response


```
{
 "subscriptionId": "weifubh82fg08dh23nhrd2f",
 "expires": "2024-02-26T13:31:05.220Z"
}
```


- An authorization needs to be performed before creating a new subscription
- expiry: seven (7) days (vendor-specific)

3.2.2 Receive Callback (CCM)

MANDATORY.
Details: A power plan needs to be announced/updated with all details in one single message by ESA Cloud.

Example Request (as received by CCM, sent by ESA Cloud)

POST https://subscribingserver.com/receivecallbacks/particularCallback


```
[
 {
  "change": "createReplace",
  "device": {
   "deviceId": "1234567890",
   "deviceType": "Washer",
   "entitiesOverview": {
    "entityTypes": [
     "Washer"
    ]
   }
  }
 },
 {
  "change": "createReplace",
  "feature": {
   "deviceId": "1234567891",
   "entityId": 0,
   "featureObjType": "powerSequence",
   "featureType": "PowerSequences",
   "data": {
    "sequenceId": 1,
    "description": "",
    "powerUnit": "W",
    "valueSource": "calculatedValue",
    "state": "scheduled",
    "activeSlotNumber": 0,
    "sequenceRemoteControllable": true,
    "startTime": "2025-04-10T18:00:00Z",
    "endTime": "2025-04-10T20:23:00Z",
    "earliestStartTime": "2025-04-10T14:24:00Z",
    "latestEndTime": "2025-04-10T20:23:00Z",
    "isPausable": false,
    "isStoppable": false
   }
  }
 }
]
```


Example Response (as received by ESA Cloud, sent by CCM)

In Case of Success:
HTTP/1.1 200

In Case of Failure:
HTTP/1.1 3xx/4xx/5xx
Content-Type: application/problem+json
Content-Language: <<isoLocale>>
Retry-After: <<delay-seconds>>

```
{
 "type": "tag:ems.miele.com,2024-09-17:example-error#correlationId",
 "title": "Internal Server Error",
 "status": 500,
 "detail": "ToDo",
 "instance": "ToDo"
}
```

Example (Failure):

HTTP/1.1 500 Internal Server Error
Content-Type: application/problem+json
Content-Language: en-US
Retry-After: 120

```
{
 "type": "tag:ems.miele.com,2024-09-17:/errors/security/authentication-error #correlationId",
 "title": "Internal Server Error",
 "status": 500,
 "detail": "ToDo",
 "instance": "ToDo"
}
```

- See Annex 2 – Error types
- The Retry-After Header indicates to ESA Cloud on when to retry a callback request on failure. ESA Cloud may apply Exponential Backoff with Jitter to prevent too much load on CCM Cloud for retries.

3.2.3 Error handling for subscriptions and callbacks

Callback from ESA Cloud to EMS-Cloud:

- If unsuccessful, retries will be attempted by the ESA Cloud based on the error code provided by the CCM (refer to 0Annex 2 – Error types). If not specified, no retries will occur (i.e., no self-healing error).
- The Retry-After Header will be set by CCM according to Annex 2 – Error types.
- Exponential backoff with jitter may be used by ESA Cloud for the retry mechanism if specified for the error code.

Requests from CCM to ESA Cloud:

- Retries will be attempted based on the error code (refer to Annex 2 – Error types). If not specified, no retries will occur (i.e., no self-healing error).
- The Retry-After Header will be set by ESA Cloud according to Annex 2 – Error types.

Deactivation of a subscription terminated by the ESA Cloud:

- Be aware that subscriptions will expire or may also be terminated by the ESA Cloud in case of many unsuccessful error retries.

4 Common workflow

Common workflow from EMS cloud perspective:

1. General access
   a) Get access to ESA cloud interface: Obtain Client Credentials for OAuth2.0 procedure
   b) Get/renew access tokens
2. Get overview on available devices, entities
   a) Subscribe to devices, entities
   b) GET on /devices, /entities
3. Get overview on (potential or available) use case-actor support
   a) Subscribe to usecases
   b) GET on /usecases
4. Get data of available use case instances
   a) Note: Data of item 3 yields if and which use case instances are available. This includes
   b) available paths (pattern /usecaseInterfaces/{usecaseName}/{usecaseMajorVersion})
      i) available address details (deviceId, entityId; evaluate isOnThisEntity and childEntities to determine the right entityId).
   c) For each use case instance of interest,
      i) subscribe to the proper usecaseInterfaces sub-path and address
      ii) GET the proper usecaseInterfaces sub-path and address
5. Control available (and controllable) use case instances:
   a) Request binding for the particular use case instance.
   b) Depending on the use case instance you need to control, consider the specification for specific operations.
      i) For FSWG, a POST operation on /usecaseInterfaces/flexibleStartForWhiteGoods/v1 is defined that can contain one of the options shiftSelectSequence or configureSequence.
6. Renew subscriptions and bindings in time.

Annex 1 – Mandatory Implementation Requirements

1) Scope: Summary of primary requirements for Use Case FSWG (Flexible Start of Whitegoods) via SPINE-IoT according to EN50631 For a given appliance supporting FSWG, at most one CCM can be given FSWG control.
   a) Consequence: Binding permission to the respective Use Case instance can be given to at most one authorized account.
      i) Exception (unlikely; just for the sake of completeness): For some reason a user has redundant EMS instances (with individual identities) and ensures that these EMS instances are synchronized/do not interfere with each other AND the cloud provider provides such configuration. This exception is NOT considered further.
   b) Consequence: IF the Use Case interface to this appliance is available for different accounts, the ESA cloud must ensure that only one account can get binding permission.
   c) Consequence: Without explicit binding messages, the ESA cloud must avoid by other means that control via Use Case interface is possible for more than one account.

2) Discovery and data access:
   a) Discovery via GET on /devices, /entities, and /usecases shall be supported by the ESA cloud.
   b) Access via /features is optional in general.
   c) Access via /usecaseInterfaces is defined per Use Case (mandatory for FSWG: and also for further use cases that will be added to EN50631; rationale: it is expected that EMS vendors just focus on /usecaseInterfaces for interactions on use cases rather than on /features; however, this will remain possible if needed (e.g. vendor specific decision)).

3) Binding and subscription:
   a) Support of binding and subscription on /usecaseInterfaces is defined per Use Case (both mandatory for FSWG)
   b) Support of subscription on /usecases is mandatory if the appliance cloud provides Use Cases (esp. via /usecaseInterfaces). Rationale: A CCM needs to be provided with updates in case of dynamically available Use Cases (example: EV is connected/disconnected to a wallbox). Note: Subscription on /usecaseInterfaces might not be sufficient to discover potentially available Use Case instances ahead of time.
   c) Support of subscription on /devices and /entities is mandatory. Rationale: This might add necessary information to prioritize all Use Case instances according to the relevance of the devices/entities.
   d) Support of subscription on /features is optional.

4) Alternative power sequences, stopping and pausing of a power sequence:
   a) More than one (alternative) power sequence MAY be provided for a given device/entity address.
      Note: See also item 6).
   b) An appliance MAY support or not support pausing/resuming or stopping a power sequence.

5) TODO... (A power plan needs to be announced/updated with all details in one single message)

6) powerSequenceAlternativesRelation and powerSequenceNode are mandatory (see below, "Excerpts 1")

7) NEW [2024-12-12]: Use only UTC times for absolute times in FSWG/powersequences time elements specified with "format: date-time".

Excerpts 1: "alternatives relation and node are mandatory"
Excerpts:

50631-1
8.12 UCF_Plan_With_Power_Sequences
8.12.1.2 Power Plan
Table 88 — Information content for Power Plan for Use Case “Flexible Start for White Goods” at Actor Device
Mandatory:

- Remote Controllable -> /features/powerSequenceNode, nodeRemoteControllable
- Supports Reselection -> /features/powerSequenceNode, supportsReselection
- Alternatives -> /features/powerSequenceAlternativesRelation

Table 88 — Information content for Power Plan for Use Case "Flexible Start for White Goods" at Actor Device

Element		Data Type	Value	M/R/O	Element and value rules
Remote Controllable		Boolean	(*N1)	M	If set to "false", the server does NOT PERMIT modifications by a client.
Alternatives Count		Integer		O	The current count of "alternatives" groups.
Total Sequences Count Max		Integer		O	The total number of sequences supported by the smart appliance across all "alternatives" groups in a single message.
Supports Reselection		Boolean		M	If set to true, this smart appliance permits selection of an optional power sequence to become the new preferred power sequence multiple times
Alternatives (*A2)				(*A1)	SHALL only be omitted if no power sequences are present in this group. (*A3)
	Alternatives ID	Integer	< as1#(1..1) >	M	PRIMARY IDENTIFIER of "alternatives" groups
	Power Sequence(1..)			M	

50631-3
5.3 Mapping to SPINE-IoT
5.3.4 UCF_Plan_With_Power_Sequences
5.3.4.2.2.1 General

The Power Plan uses the Feature Type “PowerSequences”. This comprises the Feature object types

- 1 Power Sequence Node
- 1 Power Sequence Alternatives Relation
- 0..m Power Sequence
- 0..n Power Time Slot

Annex 2 – Error types

Error responses from both CCM and ESA shall comply with RFC9457 (https://www.rfc-editor.org/rfc/rfc9457.html) Problem-JSON format.

Use the tag-URI scheme (https://www.rfc-editor.org/info/rfc4151) for the type-field, e.g.:

- tag:host,date:error-type#correlationId
  - e.g. tag:ems.miele.com,2024-09-17:transient-error#abc1234
- "about:blank" is the default for any error where there's no additional type information available beyond HTTP status code

The following error types are defined:

List of Errors returned by ESA Cloud

Type URI	Title	HTTP Status code [IANA]	Reference / Specification	Retry-After
/errors/transient-error	Transient Error	400	There's a temporary problem in cloud or controlling the appliance	tbd
/errors/value-out-of-range	Value out of Range	400	E.g. startTime value is out of allowed range	-
/errors/invalid-sequence-id	Invalid Sequence Id provided	400	An invalid sequence id was requested, e.g. not part of alternativesRelation	-
/errors/device-in-local-control	Device is in local control	403	Device is locally controlled by user, remote control thus temporarily blocked	15s
/errors/already-started	Device already started	403	Shifting power sequence requested, despite the device already started	-
/errors/device-alert-active	Alert is active on device	400	There's an active alert on the device which blocks the action	-

Subscription Callback Errors

Type URI	Title	HTTP Status code [IANA]	Reference / Specification	Retry-After
/errors/security/authentication-error	No or invalid authentication	401	Either no authentication information or a malformed information has been provided which	-

			prevents authenticating the client	
/errors/security/authorization-error	Authenticated client not authorized	403	The authenticated client is not allowed to perform this callback	-
/errors/not-found	Resource not found	404	The requested URL is not found	-
/errors/method-not-allowed	Method not allowed	405	The HTTP method is not allowed on this URL In detail: Which HTTP method has been provided and which are supported	-
/errors/not-acceptable	Not acceptable	406	The HTTP Accept header is not supported	-
/errors/unsupported-media-type	Unsupported Media Type	415	The HTTP Content-Type header is not supported	-
/errors/security/rate-limit	Too many requests	429	The CCM is overloaded with too many callback requests and asks the client to retry as specified in the Retry-After HTTP header	As specified in Retry-After header
/errors/internal-server-error	Internal Server Error	500	The CCM has an internal issue which prevents to process the callback	Retry with Exponential Backoff after 1s with jitter

Example:
HTTP/1.1 500 Internal Server Error
Content-Type: application/problem+json
Content-Language: en-US
Retry-After: 120

```
{
 "type": "/errors/internal-server-error",
 "title": "Internal Server Error",
 "status": 500,
 "detail": "ToDo",
 "instance": "ToDo"
}
```

Annex 3 – Frequently Asked Questions

1. What is the requested accuracy for Power Value?

   The specification does not prescribe a particular accuracy for power values yet.

2. Is it allowed to use a single slot?

   Yes - but modelling a power sequence with only a single slot is not recommended for good energy management.
   ESA Manufacturers should do their best to equip appliances with sufficient data to support energy management.

3. Is it necessary to send an updated Power Sequence to the CCM in case the device changes the estimated one during operations (due to particular parameters like unbalanced load, extra powder etc)

   Take advantage of the accuracy indications that are available in the different feature objects (power, powerMin, powerMax, valueSource) to send to the CCM as accurate Power profile as possible.
   In case the ESA is able to calculate its real PowerProfile at scheduling, it is advisable that it sends the real PowerProfile instead of a pre-calculated/estimated one.

4. How should CCM react in case it receives an updated Power Sequence that is changing the initial plan?

   The CCM has the decision about how to use a new power sequence received after the initial planning.
   This is not covered by the standard. It is the CCM provider's decision how to deal with the data received.

5. What happens in case the user manually pauses or stops the device?

   For Paused: The ESA sends an updated Power Sequence that indicates this new state (Paused).
   For Stopped: The ESA sends an updated Power Sequence that indicates this new state. (Inactive/Completed)

6. Which types of JWTs are used? (Callback, OAuth)

   EN50631 API uses two different types of JWTs for different purposes:
   - OAuth JWT: The JWT obtained during the OAuth Authorization Code Grant Flow, refer to chapter 1. This is sent from the EMS cloud to the ESA cloud with every synchronous request.
   - Callback Authentication JWT: The JWT used for authentication purposes within the callback header. This is sent from the ESA cloud to EMS cloud within every callback request. The definition of this JWT is currently work in progress.

7. Which version of TLS must be used?

   Encryption at transit with minimum TLS version 1.2 must be used.

8. What occurs and what steps should be taken if a user revokes their authentication for the EMS via the ESA app? For example, consider a scenario where the user has both a dishwasher and a washer linked to the EMS.

   If the user revokes the authentication for the EMS via the ESA app, the EMS will subsequently lose all access to both appliances via the SPINE-IoT API endpoints. An HTTP 401 error will occur when the EMS tries to call the SPINE-IoT API.

   It is not part of EN50631 nor OAuth 2.0 specification how subscriptions may be handled by the ESA after a user has been logged out. If possible, any subscriptions may be disabled by the ESA cloud.

   To regain access to both appliances, the EMS needs to re-prompt a new user login via ESA OAuth endpoints.

9. What occurs and what steps should be taken if a user wants to revoke access to only one appliance connected to the EMS through the ESA app, while retaining access to the second appliance?

   This is not specified by EN50631. However, if the ESA offers the possibility to reduce access to selected appliances via the SPINE-IoT API, it needs to make sure that only those selected devices will be returned via the device discovery endpoint, i.e. the EMS will always only see devices that it is allowed to see.
   As specified in EN50631, if a user deletes an appliance from the ESA cloud, ESA cloud will notify any clients with a delete-event via subscription-callback. If no subscription has been created by EMS, it can learn about appliances within a user's account via the device discovery endpoint.